# TP 02 — Cookies, sessions, SOP, CORS et CSP

FICHE ÉTUDIANTS —| Durée : 4 h 30 | PHP natif, HTML, JavaScript Vanilla

Fil conducteur : deux sites locaux et une API PHP, enrichis progressivement après le TP1.

## 1. Objectifs et prérequis

Au terme du TP, expliquer le maintien de l'état par cookie/session, identifier le triplet d'origine, distinguer restrictions SOP et permissions CORS, observer un préflight et mesurer les effets d'une CSP sur les ressources.

Prérequis : TP1 (GET/POST, en-têtes, codes), variables et formulaires PHP, bases de JavaScript et fetch(), DevTools (Réseau, Console, Application), PHP 8.1+ ou Docker.

## 2. Architecture locale

| Composant | Adresse | Rôle |
|---|---|---|
| **Site A** | http://127.0.0.1:8080 | PHP : cookies, connexion, API, CSP |
| **Site B** | http://127.0.0.1:8081 | HTML/JS : appels cross-origin vers A |

Les ports différents créent deux origines distinctes, même si les deux URL sont same-site. Attention : les cookies sont rattachés à l'hôte et au chemin, pas au port. Les protections Secure et le comportement cross-site de SameSite devront être expliqués mais non prouvés sur ces deux ports HTTP.

## 3. Démarrage et méthode de travail

```
Terminal 1 : php -S 127.0.0.1:8080 -t site-a
Terminal 2 : php -S 127.0.0.1:8081 -t site-b
```

Alternative : depuis app/, lancer docker compose up -d. Commencer par des dossiers vides site-a/ et site-b/ et construire les pages étape par étape ;

## 4. Partie A — Cookies HTTP

Concept : HTTP ne conserve pas l'état d'une requête à l'autre. Le serveur peut transmettre Set-Cookie ; le navigateur mémorise le cookie et le renvoie dans Cookie si les règles de portée le permettent.

1. Créer cookie.php et un lien « Créer » : envoyer un cookie tp2_theme=clair avec setcookie(), Path=/, Max-Age proche de 600 s (ou expiration équivalente), SameSite=Lax. Effectuer une redirection vers la page de lecture.
2. Afficher la valeur reçue par PHP via $_COOKIE, puis créer deux liens « Modifier » et « Supprimer ». Pour supprimer, réutiliser le même Path et fournir une date passée.
3. Inspecter en Réseau la réponse qui contient Set-Cookie et la requête suivante qui contient Cookie. Comparer à l'onglet Stockage.
4. Sur une copie expérimentale, activer HttpOnly puis tester document.cookie. Discuter Secure, Domain et SameSite : lesquelles de leurs propriétés ne sont pas démontrables en HTTP local sur deux ports ?

Questions A1–A4 : Pourquoi la valeur mise par setcookie() n'apparaît-elle pas immédiatement dans $_COOKIE ? Quelle différence entre Set-Cookie et Cookie ? Pourquoi une suppression exige-t-elle la même portée ? HttpOnly empêche-t-il le navigateur d'envoyer le cookie ?

## 5. Partie B — Sessions PHP

5. Créer login.php avec un formulaire POST. Comparer des identifiants fictifs locaux (etudiant / tp2026) ; ne pas intégrer de base de données.

6. Avant session_start(), fixer HttpOnly et SameSite=Lax pour le cookie de session ; désactiver Secure uniquement pour HTTP local. Activer session.use_strict_mode.
7. Après identification correcte, appeler session_regenerate_id(true) puis mémoriser $_SESSION["user"]. Créer profile.php protégé et logout.php qui détruit session et cookie.
8. Comparer PHPSESSID dans le navigateur avec $_SESSION côté serveur. Observer un accès anonyme à profile.php puis un accès authentifié.

Questions B1–B4 : Que contient PHPSESSID ? Où résident les données $_SESSION ? Pourquoi régénérer l'identifiant après connexion ? Quel risque subsiste en l'absence de HTTPS ?

## 6. Partie C — Same-Origin Policy

9. Sur B (port 8081), créer index.html et exécuter fetch("http://127.0.0.1:8080/api.php?mode=none"). L'API A doit répondre JSON mais sans Access-Control-Allow-Origin.
10. Comparer la Console et l'onglet Réseau. Déterminer si la requête a été envoyée et si la réponse est lisible en JavaScript.
11. Vérifier que deux ports différents définissent deux origines différentes. Observer qu'une requête simple peut partir même lorsque le script ne peut pas lire sa réponse.

Questions C1–C2 : Donner les triplets d'origine. SOP interdit-elle tout envoi cross-origin ? Pourquoi un script ne peut-il pas lire la réponse ?

## 7. Partie D — CORS et OPTIONS

12. Ajouter à api.php un mode allow n'autorisant que http://127.0.0.1:8081 via Access-Control-Allow-Origin. Comparer le résultat à mode=none.
13. Ajouter Access-Control-Allow-Methods: GET, OPTIONS et Access-Control-Allow-Headers: X-TP-Lab, Content-Type. Depuis B, émettre fetch() avec l'en-tête X-TP-Lab.
14. Repérer la requête OPTIONS avant GET. Noter Origin, Access-Control-Request-Method et Access-Control-Request-Headers, puis les en-têtes correspondants de la réponse.
15. Créer un mode wildcard utilisant Access-Control-Allow-Origin: * pour des données publiques uniquement. Expliquer pourquoi ce réglage ne convient pas à des données de compte privées.

Questions D1–D4 : CORS protège-t-il l'API contre curl ? Que vérifie le navigateur au preflight ? Quelle est la différence entre une autorisation de lecture et un contrôle d'accès serveur ? Pourquoi ne pas ajouter systématiquement Allow-Credentials: true ?

## 8. Partie E — Content Security Policy

16. Créer csp.php avec un script JavaScript inline, un script externe local /csp.js et une image distante.
17. Comparer sans CSP, Report-Only puis la politique : default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self'.
18. Décrire dans la console ce qui est bloqué, ce qui s'exécute et ce qui n'est que signalé par Report-Only. Ne pas confondre ressource absente et ressource bloquée par CSP.

Questions E1–E3 : Quel script est bloqué par script-src 'self' sans unsafe-inline ? Quelle est l'utilité de Report-Only ? CSP remplace-t-elle l'échappement des sorties et la validation côté serveur ?